

SDK update rules

▾ Android

[Changelog](#)[Important update for outdated versions of the AppMetrica SDK](#)[List of modules](#)[Excluding geo from the list of dependencies](#)[Disabling automatic collection of Ad Revenue data](#)[Getting advertising IDs](#)

> Migration guides

▾ AppMetrica SDK

[SDK integration](#)[Library features](#)[Tracking user activity](#)[Logging](#)[Usage examples](#)[Analyzing crashes](#)[AppMetrica Gradle Plugin](#)[Deferred deeplinks support](#)[Product flow and offer events](#)[Error descriptions](#)[SDK reference](#)

> Push SDK

> iOS

> Unity

> Flutter

> React Native

> Push campaigns

> Setting up data collection

[SDK Developer docs](#) / [Android](#) / [AppMetrica SDK](#) / [Tracking user activity](#)

Tracking user activity

In AppMetrica, a session is the period of time a user interacts with the app.

Background session

A background session starts with the AppMetrica SDK initialization.

If, during a background session, AppMetrica receives the `onResume()` system event for Android or the `UIApplicationDidBecomeActiveNotification` system event for iOS, a user session begins.

Any events sent before these system events refer to background sessions.

A background session can refer to:

- A period of time between clicking an app icon and displaying the app's interface.
- Start of services that accept push notifications.

User session

A user session starts when AppMetrica receives the following system events:

- For Android: `onResume()` .
- For iOS: `UIApplicationDidBecomeActiveNotification` .

A user session ends when the session timeout expires. The session timeout is counted from the moment the user minimi

Any events sent after the system events mentioned above refer to user sessions.

By default, AppMetrica considers a session new if a user returns to the app after a significant period of time after the app background mode (the user minimized the app or opened system settings).

Setting the session timeout

The session timeout is the period of time that must pass after a user minimizes the app and before they open it again. Yc the session timeout in the SDK.

- If the user opens the app again before the timeout expires, the current session continues.
- If the user opens the app again after the timeout expires, a new session starts.

By default, the session timeout is 10 seconds. The minimum acceptable value for the `sessionTimeout` parameter is 10

To change the timeout, pass the value in seconds to the `withSessionTimeout(int sessionTimeout)` method when extended library configuration.

Kotlin Java

```
// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
    // Setting the length of the session timeout.
    .withSessionTimeout(15)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)
```

Monitoring the app lifecycle

If the app has the minimum Android version set to 4.0 or later, the AppMetrica library can track its lifecycle automatically. initialization, call the `AppMetrica.enableActivityAutoTracking(Application application)` method:

Kotlin Java

```
class YourApplication : Application() {
    override fun onCreate() {
        super.onCreate()

        // Initializing the AppMetrica SDK.
        AppMetrica.activate(...)
        // Automatic tracking user activity.
        AppMetrica.enableActivityAutoTracking(this)
    }
}
```

**Note**

Automatic activity tracking only works for the main API key. When sending statistics to an additional API key, use `IReporter` interface methods.

If the minimum Android version in the app is earlier than 4.0, use the following methods: `AppMetrica.resumeSession()` and `AppMetrica.pauseSession(activity)` in the corresponding methods for your activity, which are `onResume()` and `onPause()`. When using those methods, make sure the active session is always terminated by calling the `AppMetrica.pauseSession(activity)` method. If that method is not called, the library considers the session active : regular data exchange with the service part of AppMetrica. This can lead to incorrect tracking of the session duration and consumption.

Example:

Kotlin **Java**

```
class YourActivity : Activity() {
    override fun onResume() {
        super.onResume()
        AppMetrica.resumeSession(this)
    }

    override fun onPause() {
        AppMetrica.pauseSession(this)
        super.onPause()
    }
}
```

Working with an additional API key

By sending data to an additional API key, you can control the access to statistics. Use reporters to transmit events. For more information, see [Usage examples](#).

For correct tracking, manually configure the reporters to send events about the start and the pause of the session for each API key. Use the `resumeSession()` and `pauseSession()` methods in the `IReporter` interface when implementing `onResume()` and `onPause()` for your activity:

Kotlin **Java**

```
class YourActivity : Activity() {
    override fun onResume() {
        super.onResume()
        AppMetrica.getReporter(applicationContext, ANOTHER_API_KEY).resumeSession()
    }

    override fun onPause() {
        AppMetrica.getReporter(applicationContext, ANOTHER_API_KEY).pauseSession()
        super.onPause()
    }
}
```

This helps the library accurately monitor:

- The number of active users.

- Session length.
- Frequency of app use.

If you didn't find the answer you were looking for, you can use the feedback form to submit your question. Please describe the problem in as much detail as possible. Attach a screenshot if possible.

[Contact support](#)[Suggest an improvement for documentation](#)

Was the article helpful?

 Yes

 No

[Previous](#)



[Library features](#)

L